

单位代码： 10293 密 级： 公开

南京邮电大学
专 业 学 位 硕 士 论 文



论文题目： 程序分析与反馈增强的 LLM 测试生成：
从上下文注入到执行反馈的迭代生成

学 号	1025040722
姓 名	邵家骅
导 师	示例导师
专业学位类别	软件工程
类 型	专业型硕士学位论文示例
专业（领域）	软件工程
论文提交日期	2026 年 5 月

摘要

近年来，基于 LLM 的测试生成研究已从仅依赖提示词的单轮生成，转向由程序分析、上下文选择和执行反馈共同驱动的迭代流程。本文综述了 2025 至 2026 年 ICSE、FSE 和 ASE 的八篇代表性论文，其中六篇构成测试生成方法的主线，两篇提供了关于测试判定依据质量和评测有效性的补充证据。本文归纳发现，这一方向的主要进展并不主要来自模型规模扩大，而更体现在任务相关上下文筛选、路径级约束构造和执行反馈注入上。同时，仅凭覆盖率和有效测试比例的提升，仍不足以保证测试质量。基于这些观察，本文提出面向测试判定依据质量的覆盖率引导生成，作为一个具体的后续研究方向。

关键词： LLM 测试生成，程序分析，执行反馈，测试判定依据，覆盖率

Abstract

This sample thesis reviews recent work on LLM-based test generation guided by program analysis and execution feedback. The reviewed studies show a shift from single-shot prompt engineering toward iterative workflows that combine relevant context selection, path-level constraints, coverage feedback, and repair. The discussion also shows that higher coverage alone is not sufficient to guarantee useful tests, because oracle quality and benchmark validity remain separate concerns. Based on these observations, the thesis sketches a coverage-guided generation workflow that also treats test oracle quality as an explicit optimization target.

Key Words: LLM-based test generation, program analysis, execution feedback, test oracle, coverage

目 录

摘要	I
Abstract	II
插图索引	IV
表格索引	V
专用术语注释表	VI
第一章 问题空间	1
第二章 方法分类	3
第三章 精读: Panta	5
3.1 问题设定	5
3.2 方法设计	5
3.3 实验结果与消融	6
3.4 局限	7
3.5 最小可验证原型	7
第四章 批判分析	9
第五章 创新提案: 面向判定依据质量的覆盖率引导生成	10
5.1 核心思路	10
5.2 与现有工作的区别	11
5.3 技术难点与开放问题	11
5.4 实验设计建议	11
第六章 结论	13
参考文献	14
程序清单	15
附录 B 攻读专业学位硕士期间撰写的论文	16
附录 B 攻读专业学位硕士期间撰写的论文	17
附录 B 攻读专业学位硕士期间撰写的论文	18

插图索引

图 1.1 问题分解图 1

表格索引

表 2.1 二维方法分类矩阵 3

表 2.2 主线方法对比：生成设置 3

表 2.3 主线方法对比：优势与局限 3

专用术语注释表

术语说明：

术语	简明解释
覆盖率（coverage）	测试运行时实际执行到的代码比例
分支覆盖率（branch coverage）	测试是否覆盖了 if/else 等不同分支
测试判定依据 / 断言（oracle / assertion）	测试中用于判断程序输出是否正确的检查语句
变异得分（mutation score）	人工向代码中注入小错误后，测试能检出多少错误的比例
CFG（控制流图）	表示代码内部执行路径结构的有向图
待测方法（focal method）	当前测试所针对的目标方法
提示词（prompt）	输入给 LLM 的任务说明、代码上下文和反馈信息

缩略词说明：

缩写	英文全称	中文译名
LLM	Large Language Model	大语言模型
CFG	Control Flow Graph	控制流图
MVP	Minimum Viable Prototype	最小可验证原型
API	Application Programming Interface	应用程序接口

第一章 问题空间

下表列出正文中频繁出现的软件测试术语，供非测试领域读者参考。

术语	简明解释
覆盖率（coverage）	测试运行时实际执行到的代码比例
分支覆盖率（branch coverage）	测试是否覆盖了 if/else 等不同分支
测试判定依据 / 断言（oracle / assertion）	测试中用于判断程序输出是否正确的检查语句
变异得分（mutation score）	人工向代码中注入小错误后，测试能检出多少错误的比例
CFG（控制流图）	表示代码内部执行路径结构的有向图
待测方法（focal method）	当前测试所针对的目标方法
提示词（prompt）	输入给 LLM 的任务说明、代码上下文和反馈信息

自动测试生成一直是软件工程的核心研究问题。其实用目标很明确——降低人工编写测试的成本，提高代码覆盖率，尽早暴露回归缺陷。大语言模型在代码补全任务中展现出的强大能力，使得研究者开始将其引入单元测试、回归测试和测试断言的生成。与传统符号执行或基于搜索的方法不同，LLM 路线的独特优势在于能够同时利用自然语言文档、历史测试样例和复杂 API 的使用模式。2023 至 2024 年间，已有若干工作将 LLM 应用于测试生成，但这些方法大多停留在一次性提示词驱动的阶段；进入 2025 至 2026 年，研究重心发生了明显转移——程序分析与执行反馈开始被系统性地嵌入生成流程。

不过，当测试对象从教科书级别的小函数换成真实项目中的复杂类时，一组结构性瓶颈反复浮现：与当前测试目标相关的上下文难以准确提供，复杂控制流难以被单次生成稳定覆盖，执行结果中蕴含的反馈信号未被充分利用，测试判定依据的质量和评测的有效性仍然不足。换言之，问题的核心不在于模型能否生成形式上正确的测试代码，而在于如何为生成过程注入足够精确、可验证的分析与反馈信号。

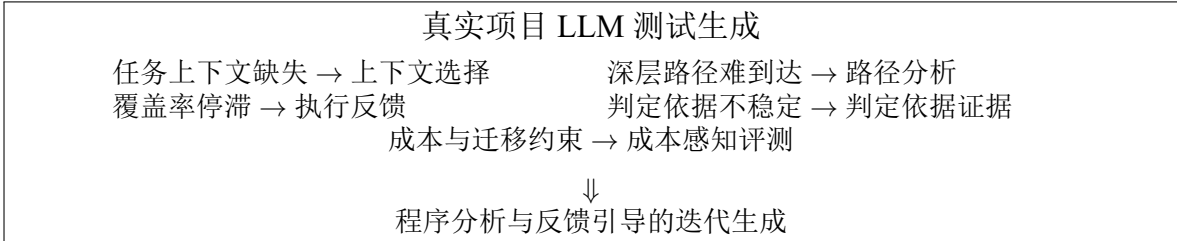


图 1.1 问题分解图

图 1. 问题分解图。真实项目中的 LLM 测试生成面临上下文、路径、反馈、测试判定依据与成本五类相互关联的约束，它们共同指向程序分析与反馈增强的迭代生成范式。

在上下文方面,困难并非缺少信息,而是难以挑出与当前测试目标相关的信息。**TestWeaver** 以动态反向切片、最近测试检索和执行状态内联三种手段,围绕未覆盖目标行组织执行条件^[1]; **KTester** 将项目结构知识、API 使用知识与测试设计知识分层注入^[2]; **JUnitGenie** 的消融实验更直接——把未经蒸馏的原始上下文送给模型,效果反而不如只给路径相关片段^[3]。三者共同说明一个判断:关键不是上下文越多越好,而是上下文越相关越好。

在路径覆盖方面,复杂控制流仍是主要障碍。**IntUT** 从 CFG 中提取条件链,并结合 mock 分类器显式构造测试意图,说明普通提示词难以恢复多层嵌套分支和 mock 约束^[4]; **Panta** 和 **JUnitGenie** 进一步表明,面对高圈复杂度方法,仅提供代码和覆盖缺口不足以支撑稳定的测试生成,还需要路径近似、路径蒸馏或目标路径排序等程序分析手段作为支架^[3,5]。

在执行反馈方面,它已从可有可无的增强项变成了主线方法的标准配置。**CoverUp** 约 40% 的成功样本来自第二轮和第三轮交互^[6]; **Panta** 将失败测试反馈和修复循环纳入核心流程^[5]; **TestWeaver** 则把执行感知机制摆在了方法的中心位置^[1]。

最后,覆盖率提升并不自动意味着测试有效性提升。**Panta** 的变异得分仅为 43.79%^[5]。**JUnitGenie** 的有效测试定义只要求测试能执行到待测方法,不保证断言正确^[3]。**Doc2OracLL** 与 **Oracle-Unbiased** 则说明,测试判定依据质量需要单独讨论,基准数据泄漏也会干扰模型能力判断^[7,8]。

综合以上四个方面,可以将该问题空间概括为:真实项目中的 LLM 测试生成无法仅靠一次性提示词稳定产生高质量测试,因此需要结合程序分析选择目标路径、筛选相关上下文,并利用执行反馈迭代修复生成结果。正因如此,2025 至 2026 年的代表性工作开始将程序分析与反馈机制系统性地嵌入生成流程。

第二章 方法分类

对这批工作做分类，按年份或模型类型来排列都不够稳定——同一年内方法差异很大，不同年份的方法又可能共享相同技术路线。更合适的方式是从两个维度入手：约束信息来源（模型依赖什么信息生成测试）和是否依赖执行反馈。本文综述的八篇论文分为两组：六篇构成测试生成方法的主线，两篇（Doc2OracLL、Oracle-Unbiased）提供测试判定依据质量与评测有效性的补充证据，不参与主方法分类，而是用于支撑第 4 节的批判分析和第 5 节的创新方向。

表 2.1 二维方法分类矩阵

	无执行反馈	有执行反馈
上下文 / 知识注入	IntUT ^[4] , KTester ^[2]	JUnitGenie ^[3]
覆盖 / 路径引导	—	CoverUp ^[6] , Panta ^[5] , TestWeaver ^[1]
判定依据 / 评测（补充证据）	Doc2OracLL ^[7] , Oracle-Unbiased ^[8]	—

表 2.2 主线方法对比：生成设置

论文	核心思路	约束来源	反馈信号
CoverUp ^[6]	覆盖率引导的迭代生成	覆盖缺口 + 代码上下文	覆盖缺口 + 执行结果
IntUT ^[4]	测试意图引导生成	测试意图 + mock 知识 + CFG 条件链	无
KTester ^[2]	项目知识引导生成	项目知识 + 测试知识	部分静态校验 + 知识引导生成
JUnitGenie ^[3]	路径敏感生成	路径蒸馏后的项目上下文	执行校验 + 路径重试
Panta ^[5]	混合程序分析引导生成	CFG 路径 + 覆盖缺口	覆盖反馈 + 编译修复 + 失败测试反馈
TestWeaver ^[1]	执行感知的反馈驱动生成	动态切片 + 最近测试 + 执行状态	动态执行状态 + 覆盖目标

表 2.3 主线方法对比：优势与局限

论文	优势	局限
CoverUp ^[6]	轻量级设计，多轮反馈稳定提升覆盖率	不讨论断言质量，仅面向 Python
IntUT ^[4]	在工业代码上分支覆盖率从 33% 提升到 64%	无迭代反馈，依赖闭源工业数据
KTester ^[2]	知识分层注入，测试风格与项目一致	修复能力有限，知识提取依赖静态分析完整性
JUnitGenie ^[3]	路径蒸馏消融实验清晰，上下文精简有效	平均每项目 13.68h，成本高
Panta ^[5]	路径分析 + 反馈闭环，复杂方法覆盖率提升最大	变异得分仅 43.79%，仅限单类
TestWeaver ^[1]	唯一面向回归测试，执行感知输入	覆盖率数字在摘要与表格间有口径差异

注：表中数字只表示各论文内部报告的代表性结果，不能直接视为跨论文公平排名。

从约束信息来源看，六篇主线论文的输入策略各有侧重。CoverUp 的策略最轻量——直接以覆盖缺口定位待生成区域，并通过 get_info 工具按需拉取代码上下文，不做重度预处理

理^[6]。IntUT 走的是显式意图构造路线：它从 CFG 中提取分支到达条件链，结合 mock 分类器判断哪些依赖需要 mock，最终把这些结构化信息打包为测试意图输入模型^[4]。KTester 则站在项目知识的视角，先用静态分析从仓库中抽取类结构、方法调用关系和 API 使用模式，再将项目知识与测试设计知识分两步注入生成过程^[2]。JUnitGenie 比 KTester 更进一步，不仅抽取项目级信息，还对其进行路径敏感的蒸馏——只保留与当前测试目标路径相关的依赖和类型签名，丢弃无关片段^[3]。Panta 在此基础上加入了程序分析的硬约束：它从方法级 CFG 中提取线性无关路径，并以 missedScore 排序来决定每轮优先推进哪条未覆盖路径^[5]。TestWeaver 的输入则最具运行时色彩——它对目标行做动态反向切片，检索与该行最相似的已有测试，并将执行轨迹中的变量值内联回代码上下文，形成一种执行感知的输入表示^[1]。

从测试目标看，前五篇都以单元测试为主要生成对象，但优化重点不同。CoverUp 和 Panta 追求高覆盖率，IntUT 追求对开发者意图的准确表达，KTester 追求与项目惯例一致的测试风格，JUnitGenie 追求可执行的路径敏感测试。TestWeaver 是六篇中唯一面向回归测试的，它关注的不是从零开始生成测试，而是在已有测试基础上扩展覆盖范围。Doc2OracLL 和 Oracle-Unbiased 不生成完整测试，只关注测试判定依据的生成与评测^[7,8]。

按研究演化来看，这条主线经历了几个清晰的阶段（CoverUp → IntUT / KTester → JUnitGenie → Panta / TestWeaver）。CoverUp 率先证明了多轮反馈的价值；IntUT 和 KTester 开始向提示词中注入结构化约束；JUnitGenie 引入路径敏感的上下文蒸馏；Panta 和 TestWeaver 把程序分析、执行反馈与修复接成了完整闭环。在本文综述的工作中，这些提升分别来自上下文筛选^[2,3]、路径级约束^[3,5]和执行反馈^[1,5,6]，而不是单纯依赖模型规模扩大。

上述分类反映出三个稳定趋势。第一，性能较好的方法普遍采用显式的约束组织机制，而非依赖模型自行从长上下文中恢复目标。第二，执行反馈与修复已成为主线方法的共性组成。第三，Panta 和 JUnitGenie 显示覆盖率或有效测试比例的提升并不保证强断言^[3,5]；Doc2OracLL 和 Oracle-Unbiased 则进一步说明测试判定依据质量需要单独评估^[7,8]。

第三章 精读：Panta

在这批工作中，Panta 最适合作为精读对象。原因有二：它是目前将路径级程序分析、覆盖反馈和测试修复接成完整闭环的最完整实例；它的消融实验设计也比较充分，能清楚说明各个组件的实际贡献。

3.1 问题设定

Panta 的出发点很具体：在 Defects4J 数据集中，当待测方法的圈复杂度较高时，现有 LLM 方法（包括已经采用覆盖反馈的方法）的覆盖率会显著下降。作者的判断是，这种性能退化的根源不在模型的代码生成能力，而在于模型无法自主识别应当推进的未覆盖路径。一次提示词能覆盖的往往只是容易到达的分支；对于嵌套条件、多步状态转换、需要特定前置条件才能触发的深路径，模型既缺少路径信息，也缺少失败后的修正信号^[5]。

3.2 方法设计

Panta 的生成流程包含四个环节，它们之间形成迭代关系而非线性流水线。

第一步：路径提取。对每个待测方法构建方法级 CFG（控制流图），然后从中提取一组线性无关路径。这里的线性无关指图论意义上的性质：每条路径至少经过一条其他路径未覆盖的边。这组路径并非精确的执行路径（精确枚举在一般情况下不可行），而是对方法内部分支结构的一种近似覆盖。算法保证每个待测方法提取的路径数等于其圈复杂度，而不是枚举所有可能路径^[5]。

第二步：路径排序与目标选择。运行当前测试套件，收集行和分支级别的覆盖数据。对于每条未覆盖路径，Panta 计算一个 missedScore，即该路径上仍未被任何现有测试覆盖的语句和分支的计数加和。missedScore 越高，说明这条路径对提升覆盖率的潜在贡献越大。路径选择不仅看 missedScore，还维护历史选择次数（pathHistory），在优先选择高分路径和优先尝试低频路径之间平衡，并限制对反复无效路径的重试次数。每轮综合两个信号选出本轮生成目标^[5]。

第三步：上下文组装与生成。将以下信息拼装为提示词输入模型：待测方法源码、当前已有测试、项目依赖签名、目标路径描述、以及上一轮失败测试的错误信息（如果有的话）。这里有一个值得注意的设计选择——Panta 不把整个类或整个文件送给模型，而是只提供方法级别的上下文加上路径级别的约束，这与其他方法（如 KTester 的项目级知识注入）形成对照^[5]。

第四步：验证、修复与反馈回注。执行生成的测试。通过编译和运行的测试直接加入测试套件；编译失败的测试进入修复流程（将错误信息反馈给模型重新生成）；运行失败但可编译的测试则作为失败测试反馈保留到下一轮提示词中。整个过程在满足停止条件（如达到迭代上限或覆盖率不再增长）前持续运行^[5]。

Algorithm: Core Panta Loop

Input: source file S , existing tests T

```

1: methodDict <- approximatePaths(S)
    // extract linearly independent paths for each method under test
2: pathHistory <- dict()
3: failedTestFeedback <- []
4: coverage <- runTestsWithCoverage(T)
5: while iter < maxCYC and noIncrease < maxNoIncreaseLimit
    and coverage < 100%:
6:     selectedPaths <- {}
7:     for each method in methodDict:
8:         candidatePaths <- scoreUncoveredPaths(method, coverage)
9:         highestPath <- max(candidatePaths, key = missedScore)
10:        leastPath <- min(candidatePaths, key = pathHistory)
11:        selectedPaths.add(highestPath, leastPath)
12:        update pathHistory for selected paths
13:    prompt <- buildPrompt(S, T, selectedPaths, failedTestFeedback)
14:    generatedTests <- LLM(prompt)
15:    for each test in generatedTests:
16:        result <- addTestAndRun(test, T)
17:        if result.failed:
18:            removeTest(test, T)
19:            testsToRepair.add(test, result.error)
20:    failedTestFeedback <- repairAndValidate(testsToRepair)
21:    newCoverage <- runTestsWithCoverage(T)
22:    update noIncrease, coverage, and iter

```

3.3 实验结果与消融

在 Defects4J 的复杂类设定下（筛选圈复杂度较高的类），Panta 相对 SymPrompt 基线的提升幅度为：行覆盖率 43.91%→70.18% (+26.3pp)，分支覆盖率 38.17%→60.83% (+22.7pp)，通过率 32.83%→55.92%，变异得分 31.30%→43.79%。

消融实验的结果对理解方法贡献更有价值。去掉路径分析只保留覆盖反馈时，覆盖率仍有提升但幅度明显缩小——完整 Panta 相比仅覆盖率引导版本仍能取得约 10 个百分点的额外

增益。这说明在复杂代码场景下，仅靠告诉模型哪些行没覆盖是不够的，还需要指明沿着哪条路径去覆盖。路径级程序分析是 Panta 区别于 CoverUp 等纯覆盖引导方法的核心差异^[5]。

3.4 局限

这篇论文同样清楚地暴露了当前路线的边界。第一，Panta 只在单类上下文中工作，跨类依赖、继承层次和框架环境（如 Spring）会削弱路径提取的完整性。第二，修复环节对编译错误的修复效果较好，但对 AssertionError 这类语义错误基本无能为力——模型不知道正确的预期值应该是什么。第三，变异得分仍低于 50%，这意味着即使覆盖率大幅提升，生成的测试在检出人工注入缺陷方面的能力仍然有限^[5]。

3.5 最小可验证原型

若以最小可验证原型复现 Panta 的核心思想，合理的范围是保留三个环节：方法级 CFG 路径提取（可基于 Soot 或 JavaParser 实现）、基于 missedScore 的覆盖驱动目标选择，以及失败测试回注。修复环节可先限定在编译错误修复，不必一开始就处理断言语义修正。MVP 的核心实验不是复现 Panta 全部效果，而是比较两组设置：仅覆盖率引导（coverage-only）与路径加覆盖率联合引导（path+coverage）。若后者在高圈复杂度方法上显著提升分支覆盖率，则说明路径引导的核心假设成立。

Algorithm 2: Minimal Path-and-Coverage Scheduler

Input: focal method m , current tests T , extracted paths P ,
coverage report C , retry budget B

```

1: for each path  $p$  in  $P$ :
2:     missedScore[ $p$ ]  $\leftarrow$  uncovered_lines( $p$ ,  $C$ ) + uncovered_branches( $p$ ,  $C$ )
3:     priority[ $p$ ]  $\leftarrow$  missedScore[ $p$ ] -  $\lambda$  * attempts[ $p$ ]
4: target  $\leftarrow$  argmax $_p$  priority[ $p$ ]
5: prompt  $\leftarrow$  build_prompt(source( $m$ ), path_conditions(target),
    uncovered_lines(target), existing_tests( $T$ ), last_failure_log)
6: candidates  $\leftarrow$  LLM(prompt)
7: for each test  $t$  in candidates:
8:     result  $\leftarrow$  run_test( $t$ )
9:     if result.compiles and result.passes:
10:         C_new  $\leftarrow$  measure_coverage( $T$  union { $t$ })
11:         if branch_coverage(C_new) > branch_coverage( $C$ ):
12:             T  $\leftarrow$  T union { $t$ }
13:     else if result.compile_error and attempts[target] < B:
14:         last_failure_log  $\leftarrow$  result.error_message

```

```
15:         attempts[target] <- attempts[target] + 1
16: return T
```

这个 MVP 不追求复现完整 **Panta** 系统，而是隔离一个核心假设：在相同 LLM 和执行预算下，路径加覆盖率联合调度应当比仅覆盖率调度更适合高圈复杂度方法。

第四章 批判分析

前三节表明，程序分析与反馈确实能提升覆盖率和可执行性。但这并不意味着问题已经解决。以下从四个角度讨论现有工作尚未充分回答的问题。

覆盖率只是弱代理指标。 CoverUp 的主实验几乎完全围绕覆盖率展开^[6]，JUnitGenie 对有效测试的定义只要求测试执行到待测方法，不保证断言正确^[3]，Panta 虽然额外报告了变异得分，但均值仅为 43.79%^[5]。这些结果说明，主线论文在测试更容易运行和覆盖率更高两方面有较强证据，但在断言更准确和缺陷检出更好方面，证据明显薄弱。

上下文不是越多越好。 JUnitGenie 的消融实验直接给出了反例——不经蒸馏的原始上下文导致分支覆盖率从 56.86% 退到 34.44%^[3]。TestWeaver 用切片和最近测试替代整段程序上下文^[1]，KTester 先对项目知识做结构化整理再注入^[2]。Oracle-Unbiased 从测试判定依据生成的角度给出进一步的佐证：在提示词中加入额外的待测类和测试类上下文，收益不到 1%^[8]。仓库感知方法的真正难点不在于获取更多信息，而在于排除无关信息。

反馈有效但代价不低。 CoverUp 约 40% 的成功样本来自二三轮交互^[6]，TestWeaver 报告了明确的时间与美元成本（0.43 小时 / \$3.93 达到 62% 覆盖率）^[1]，JUnitGenie 平均每项目耗时 13.68 小时且大部分时间花在修复阶段^[3]。反馈闭环确实有效，但它同时把问题转化为一个工程成本问题——多轮交互、反复执行、反复修复带来的开销是否可接受，取决于具体应用场景。

评测条件限制了结论的可推广性。 这一局限可以从三个层面拆开讨论。

数据集外部有效性。 IntUT 的主实验基于闭源工业数据，外部可核验性较弱^[4]；其他论文主要在 Defects4J 等 Java 基准或 Python 开源项目上验证，对工业闭源代码、混合语言项目的泛化能力尚不明确。

指标外部有效性。 不同论文采用的核心指标（覆盖率、有效测试比例、变异得分）衡量的是不同维度的测试质量。TestWeaver 的关键覆盖率数字在摘要与表格之间存在口径差异（摘要报告行分支覆盖率 62%，但分项数字分别为行覆盖率 68% 和分支覆盖率 54%）^[1]；KTester 的表 2 与正文表述不完全一致^[2]。这些差异虽不一定影响核心结论，但提示读者在横向比较时应注意指标定义的统一性。

训练数据泄漏。 Oracle-Unbiased 揭示了一个更根本的问题：在时间截止点之前的基准数据可能已经进入了模型的训练集，导致测试判定依据生成能力被高估^[8]。当前研究不仅存在方法边界，也存在证据边界。

第五章 创新提案：面向判定依据质量的覆盖率引导生成

前面的分析反复指向同一个缺口：主线论文在覆盖率和可执行性方面取得了实质进展，但测试判定依据质量始终没有被以同等强度建模。Panta 的变异得分仅 43.79%，CoverUp 几乎不讨论断言质量，JUnitGenie 的有效测试定义不要求测试判定依据正确。与此同时，Doc2OracLL 说明文档信号有独立价值；加入 Javadoc 后，准确率从 57.95% 提升到 78.11%^[7]。Oracle-Unbiased 则要求测试判定依据评测必须排除泄漏数据^[8]。两类工作之间缺少一个桥梁——将测试判定依据质量反馈嵌入覆盖引导的迭代生成流程。

5.1 核心思路

这一方向的目标是将现有的覆盖率引导循环扩展为覆盖率与测试判定依据并重的双目标闭环。具体而言，每轮迭代包含四步：

1. **目标选择**：沿用 Panta 式的路径分析或 CoverUp 式的覆盖缺口分析，选出本轮应推进的未覆盖目标。
2. **判定依据线索提取**：从文档（如 Javadoc 的 @return、@throws）、历史测试中的断言模式、以及相邻方法的已有测试中提取判定依据线索，拼入提示词。
3. **生成与双重验证**：生成测试后，不仅检查是否通过编译和执行，还检查断言是否被弱化（例如是否退化为 `assertNotNull` 或空断言体），是否仅实现了表面覆盖。
4. **判定依据级反馈回注**：如果断言质量不达标，将判定依据级失败（而不仅仅是执行失败）作为反馈信号输入下一轮生成。

候选测试的接受条件可以写成一个双目标打分函数：

```
score(t) =  
    alpha * branchCoverageGain(t)  
+ beta * oracleStrength(t)  
- gamma * cost(t)
```

```
oracleStrength(t) =  
    killedMutants(t)  
- weakAssertionPenalty(t)  
+ documentationConsistency(t)
```

系统不应仅因为一个测试提高覆盖率就保留它；候选测试至少应提高分支覆盖率，或在不引入弱断言的前提下增强判定依据证据。

5.2 与现有工作的区别

这一方向的新增贡献不在于设计新的提示词模板，而在于改变闭环的反馈信号。现有方法的反馈信号主要是测试是否通过编译和执行，以及哪些行还没被覆盖；这里额外引入的信号是断言是否足够强。这一区别看似微小，但它直接影响系统的优化方向——如果反馈只看覆盖率，系统会倾向于生成能跑通但断言空洞的测试；加入测试判定依据质量约束后，系统被迫在覆盖率和断言可验证性之间寻找平衡。

5.3 技术难点与开放问题

这一方向面临两个核心挑战。第一，文档和历史断言本身可能包含噪声。如果 Javadoc 的描述不准确或过时，从中提取的判定依据线索会把错误信号引入闭环，导致生成质量反而下降。当多个来源发生冲突时（例如文档说返回空列表，历史测试却断言返回 null），系统需要一套可信度裁决机制——一个可行的策略是按来源可靠性排序：通过的测试断言 > 文档描述 > 相邻方法推断，并在冲突时向模型同时暴露矛盾信息，由模型生成多个候选断言再通过执行验证筛选。第二，测试判定依据质量的自动判定本身就是难题。建议定义三类可操作的质量指标：

指标	含义
断言密度	每个测试方法中显式断言的数量
弱断言比例	assertNotNull、空断言体、无条件通过等弱断言占比
变异得分	测试能杀死多少人工注入变异体的比例

如果测试判定依据校验器仍然依赖精确字符串匹配，那就会重复 Oracle-Unbiased 已经批评过的评测局限^[8]。一个可能的折中方案是利用变异测试作为弱校验器——如果一个断言不能杀死任何变异体，它很可能缺乏辨别力。但变异测试本身的计算成本不低，如何在判定依据检验的可信度与生成流程的效率之间取得平衡，仍然是开放问题。

5.4 实验设计建议

为使上述提案具备可评测性，建议采用以下实验对照设计：

设置	说明
仅覆盖率引导	模拟 CoverUp / Panta 式纯覆盖率优化，不考虑断言质量
仅断言质量引导	只优化断言质量，不引导覆盖路径
覆盖率 + 判定依据	本文提出的双目标闭环方法
人工编写测试	作为参考上界或质量对照

核心评测指标包括分支覆盖率、断言密度、弱断言比例和变异得分。若覆盖率 + 判定依据组在变异得分上显著优于仅覆盖率引导组，同时分支覆盖率不显著退化，则说明判定依据感知反馈确实能在不牺牲覆盖率的前提下提升测试质量。

此外值得注意的是，当前所有方法（包括本提案）都面临一个共性的工程约束：多轮迭代、执行反馈与上下文检索带来的 token 和时间成本。TestWeaver 和 JUnitGenie 的实验已经表明这类成本不可忽视^[1,3]。后续工作如果要使判定依据感知生成具备实用价值，将检索预算和迭代次数纳入系统目标，即走向成本感知的生成策略，将是自然的延伸方向。

第六章 结论

综合全文来看，2025 至 2026 年的 LLM 测试生成研究已从模型直接生成测试，转向由程序分析、上下文选择与执行反馈共同约束的迭代生成。本文综述的八篇工作表明，结构化约束与迭代反馈能够切实提升覆盖率与可执行性，但测试判定依据质量仍是尚未被充分回应的核心问题。将测试判定依据质量约束显式嵌入生成闭环，是从生成更多测试走向生成更有用测试的关键一步。

作为格式检查样例，本文已经填充封面、摘要、目录、图表索引、术语页、正文、参考文献以及后置占位信息。文中方法细节和实验数字以原始初稿为来源，当前版本不用于事实核验，只用于检查 NJUPT 学位论文模板的排版效果。

参考文献

- [1] Le C C, Van C D, Vu T D, et al. Testweaver: Execution-aware, feedback-driven regression testing generation with large language models[C]. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE). ACM, 2026.
- [2] Li A, Liu M, Chen Z, et al. Knowledge matters: Injecting project and testing knowledge into llm-based unit test generation[C]. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE). ACM, 2026.
- [3] Liao D, Yin X, Pan S, et al. Navigating the labyrinth: Path-sensitive unit test generation with large language models[C]. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). [S.l.]: IEEE, 2025: 687–699.
- [4] Nan Z, Guo Z, Liu K, et al. Test intention guided llm-based unit test generation[C]. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025: 1026–1038.
- [5] Gu S, Nashid N, Mesbah A. Llm test generation via iterative hybrid program analysis[C]. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE), Research Track. [S.l.: s.n.], 2026.
- [6] Pizzorno J A, Berger E D. Coverup: Effective high coverage test generation for python[J]. Proceedings of the ACM on Software Engineering, 2025, 2(FSE): FSE128.
- [7] Hossain S B, Taylor R, Dwyer M. Doc2oracll: Investigating the impact of documentation on llm-based test oracle generation[J]. Proceedings of the ACM on Software Engineering, 2025, 2(FSE): 1870–1891.
- [8] Molinelli D, Di Grazia L, Martin-Lopez A, et al. Do llms generate useful test oracles? an empirical study with an unbiased dataset[C]. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). [S.l.]: IEEE, 2025: 278–290.

程序清单

本示例论文不包含实际软件系统实现。为检查模板后置部分格式，保留一个最小伪代码条目。

Input: focal method, current tests, coverage report

Output: generated tests for uncovered paths

1. extract path candidates from the control-flow graph
2. rank candidates by uncovered lines and branches
3. ask the LLM to generate tests for the selected path
4. run tests and feed failures back to the next iteration

附录 B 攻读专业学位硕士期间撰写的论文

- [1] 示例作者. 程序分析与反馈增强的 LLM 测试生成综述 [J]. 示例期刊, 2026, 1(1):1-10.
(示例条目)

附录 B 攻读专业学位硕士期间撰写的论文

- [1] 示例作者. 一种覆盖率与测试判定依据联合引导的测试生成方法 [P]. 中国, 202610000000.0. (示例条目)

附录 B 攻读专业学位硕士期间撰写的论文

- [1] LLM 测试生成格式检查示例项目，2026.（示例条目）